Machine Learning Supervisionado

Aula 05 — Da regressão linear ao XGBoost

Regressão Linear • Árvore de Decisão • Random Forest • XGBoost

Prof. Luis Vasconcelos | Duração: 100 min

Ensinando o Computador a Prever

Se eu te disser T_top = 78,5 °C, quanto você chuta que é a composição?

Chute manual

Baseado na intuição

Visualmente:
Reta suave
vs.
Degraus retos

Regressão Linear

Reta otimizada (MSE mínimo)

Visualmente:
Reta suave
vs.
Degraus retos

Árvore de Decisão

Regras binárias em degraus

Visualmente:
Reta suave
vs.
Degraus retos

"Modelo fenomenológico: y = f(X) de leis físicas. ML: y = f(X) aprendida dos dados."

Pergunta Desafio

"Se uma árvore de decisão pode desenhar degraus para ajustar qualquer curva, por que não usamos árvores gigantes com profundidade 1000?"

💡 Pense: a árvore memoriza os dados de treino? E quando chega um dado novo?

Quatro Algoritmos, Uma Missão: Predizer

Regressão Linear

$\hat{y} = w_1x_1 + w_2x_2 + ... + b$

Encontra pesos que minimizam
o erro quadrático (MSE).

Interpretação física:
cada w_i = impacto de x_i em y

Árvore de Decisão

Fluxograma de perguntas binárias.

Ex: T_top > 78,5°C?
Sim → região A (média = 0.88)
Não → região B (média = 0.90)

Predição = valor médio da região

Random Forest

Muitas árvores treinadas em
dados aleatórios (bootstrap).

Predição = média de TODAS
as árvores.

Reduz overfitting.

XGBoost

Árvores treinadas SEQUENCIALMENTE.

Cada árvore nova corrige
os erros da anterior.

Gradient descent no espaço
de modelos.

Analogia: Cada Modelo É uma Ferramenta Diferente

Regressão Linear	Traça uma reta reta. Simples, interpretável, mas limitada.
Árvore de Decisão	Recorta regiões retangulares. Flexível, mas pode overfit.
Random Forest	Muitos estênceis votam. Robusto, generalize bem.
XGBoost	Cada membro aprende com o erro do anterior. Poderoso.

Na engenharia química: escolha a ferramenta certa para o problema certo.

Pipeline de ML em 6 Passos

①	**Split**	`train_test_split(X, y, test_size=0.2)`
②	**Instanciar**	`model = LinearRegression()`
③	**Treinar**	`model.fit(X_train, y_train)`
④	**Predizer**	`y_pred = model.predict(X_test)`
	Avaliar	`r2_score(), mean_squared_error()`
⑥	**Comparar**	todos os modelos lado a lado

$$MSE = (1/n)\ \Sigma(y_i - \hat{y}_i)^2 \quad | \quad R^2 = 1 - SS_res\ /\ SS_tot$$

◌ Prática: Treinando 4 Modelos

Soft-sensor: predizer composição do destilado em tempo real!

- Carregue coluna_destilacao_30dias.csv
- Separe X (7 features) e y (composicao)
- Split 80/20 com random_state=42
- Treine: LinearRegression, DecisionTree, RandomForest, XGBoost
- Calcule R^2 e RMSE para treino e teste
- Crie tabela comparativa + plot predicted vs real

```
modelos = {
    'Linear': LinearRegression(),
    'Árvore': DecisionTreeRegressor(max_depth=5),
    'RF': RandomForestRegressor(n_estimators=100),
    'XGB': XGBRegressor(n_estimators=100, lr=0.1)
}
```

Diagnóstico de Overfitting por Hiperparâmetro

Grupo A	Grupo B	Grupo C	Grupo D
max_depth (Árvore) Valores: 3, 5, 10 Para cada valor: 1) R^2 treino 2) R^2 teste 3) Gap = overfitting	n_estimators (RF) Valores: 10, 100, 500 Para cada valor: 1) R^2 treino 2) R^2 teste 3) Gap = overfitting	max_depth (RF) Valores: 3, 10, None Para cada valor: 1) R^2 treino 2) R^2 teste 3) Gap = overfitting	learning_rate (XGB) Valores: 0.01, 0.1, 0.3 Para cada valor: 1) R^2 treino 2) R^2 teste 3) Gap = overfitting

🧠 Pergunta: Qual valor minimiza o gap treino-teste? Qual maximiza R^2 de teste?

Random Forest vs XGBoost

Random Forest — Paralelo

- Árvores treinadas INDEPENDENTEMENTE
- Cada árvore vê dados diferentes (bootstrap)
- Predição = média de todas as árvores
- Reduz variância, robusto a overfitting
- Menos sensível a hiperparâmetros

XGBoost — Sequencial

- Árvores treinadas SEQUENCIALMENTE
- Cada árvore corrige erros da anterior
- Minimiza perda via gradient descent
- Mais preciso, mas mais sensível
- Requer tuning de learning_rate

Na engenharia química: RF = seguro e robusto | XGBoost = máximo desempenho com tuning.

Tarefa: Comparando Modelos para o CSTR

- Use o dataset CSTR (Aula 3): features + conversão
- Treine os 4 modelos (Linear, Árvore, RF, XGBoost)
- Calcule R^2 e RMSE (treino e teste)
- Identifique: maior overfitting? melhor generalização?
- Plote predicted vs real (melhor e pior lado a lado)
- Escreva 1 parágrafo: qual modelo implementaria como soft-sensor?
- Entregue via Colab — Prazo: antes da Aula 6 (7 dias)

Bônus NT: Treine sem a feature T_reator — o R^2 cai quanto? Modelo ainda útil? (+20%)

Próxima aula: Avaliação e Validação de Modelos — R^2, RMSE, MAE, validação cruzada. 🚀